Studio Nx

Backend(FastAPI/Python) 12 — Track 모델 컬럼 (Phase 1 칸)

2026-10-12

Backend(FastAPI/Python) 12 — `Track` 모델 컬럼 (Phase 1 칸만)

항목	내용
파일	`studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	11 `ProjectMember` 모델 · 13 관계

1. 하는 일

MySQL의 `tracks` 테이블(시퀀서의 트랙 한 줄)을 파이썬 클래스 `Track`으로 옮겨 적습니다. 테이블에는 칸이 19개 있지만, **Phase 1에 필요한 3칸만** 채웠습니다. 나머지 14칸(색, 볼륨, 음소거 등)은 시퀀서를 만드는 Phase 2에서 채웁니다.

지금 모델에 있는 칸	뜻
`track_id`, `project_id` (뼈대에 있었음)	트랙 번호와 트랙이 속한 프로젝트입니다
`name`	트랙 이름입니다. 예: `Master`
`track_type`	트랙 종류입니다. `AUDIO`, `MIDI`, `MASTER`, `GROUP`, `RETURN` 중 하나입니다
`order_index`	화면에서의 순서입니다 (0부터)

2. 언제 쓰나

Phase 1에서는 **프로젝트를 만들 때 MASTER 트랙 1개를 넣는 데만** 씁니다 (P1-13, FR-122). studioNx.html을 처음 열면 맨 아래에 보이는 Master 트랙이 바로 이 행입니다.

```
# P1-13 project_service.create_project() 안 (예정)
db.add(Track(project_id=project.project_id, name="Master", track_type=TrackType.MASTER, order_index=0))
```

프로젝트를 만들 때 함께 넣는 것	모델
프로젝트 행	10 `Project`
OWNER 멤버 행	11 `ProjectMember`
MASTER 트랙 행	**12 `Track`**

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, ForeignKey, Integer, String, text
from sqlalchemy.orm import Mapped, mapped_column
```

```
from app.db.session import Base
```

`BigInteger`, `ForeignKey`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, **`StrEnum`, `Enum`, `Integer`, `String`, `text`**를 이번에 추가했습니다. 모두 이 클래스에서 씁니다. 시각 칸(`created_at` 등)을 아직 넣지 않았기 때문에 `DATETIME`은 없습니다.

함수 (클래스와 칸 선언)

```
class TrackType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"
    MASTER = "MASTER"
    GROUP = "GROUP"
    RETURN = "RETURN"


class Track(Base):
    __tablename__ = "tracks"

    track_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )

    # Phase 1 칸 (MASTER 트랙 생성용). 나머지 칸은 DB 기본값 · NULL 로 채워진다
    name: Mapped[str] = mapped_column(String(100))
    track_type: Mapped[TrackType] = mapped_column(Enum(TrackType))
    order_index: Mapped[int] = mapped_column(Integer, server_default=text("0"))

    # TODO(Phase 2): color, volume_db, pan, is_muted, is_solo, … (뼈대의 주석 그대로)
```

SQL과 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`name VARCHAR(100) NOT NULL`	`Mapped[str]`, `String(100)`
`track_type ENUM('AUDIO',…,'RETURN') NOT NULL` (**기본값 없음**)	`Enum(TrackType)`이고, `server_default`가 없습니다
`order_index INT UNSIGNED NOT NULL DEFAULT 0`	`Integer`, `server_default=text("0")`
`color CHAR(7) NOT NULL DEFAULT '#7C5CFF'` 외 13칸	적지 않았습니다 (Phase 2). 행을 넣을 때 DB가 기본값이나 `NULL`로 채웁니다

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
모델에 없는 칸	SQLAlchemy는 모델에 적힌 칸만 `INSERT`하고 `SELECT`합니다. 나머지 칸은 DB가 기본값으로 채우고, 파이썬에서는 `track.color`처럼 읽을 수 없습니다

식	뜻
`track_type`에 `server_default`가 없음	반드시 넣어야 하는 칸입니다. 빠뜨리면 DB가 거절합니다

4. 흐름 (MASTER 트랙을 넣을 때)

1. `Track(project_id=12, name="Master", track_type=TrackType.MASTER)`를 만듭니다.
2. `db.add` → `db.flush()`로 `INSERT INTO tracks (project_id, name, track_type) …`를 실행합니다.
3. DB가 `track_id`, `order_index=0`과 모델에 없는 칸(`color='#7C5CFF'`, `volume_db=0.00`, `is_muted=0`, `routing_mode='CHAIN'` …)을 채웁니다.

5. 왜 이렇게

선택	이유
Phase 1 칸만 채움	뼈대의 TODO가 Phase 1 칸만 정해 두었습니다. 나머지 칸은 시퀀서(Phase 2)에서 의미가 정해지므로, 지금 적으면 쓰지 않는 코드가 됩니다. 모두 DB 기본값이나 `NULL`이 있어서 행을 넣을 때 문제가 없습니다
`track_type`에 기본값을 두지 않음	DB에도 기본값이 없습니다. 트랙 종류는 만드는 쪽이 반드시 정해야 합니다
FK는 `project_id` 하나만 적음	DB에는 FK가 5개 있지만, 나머지 4개(`parent_track_id`, `frozen_sample_id`, `assignee_id`, `locked_by`)는 Phase 2 칸에 붙어 있습니다. 그 칸을 채울 때 함께 적습니다
색인(`idx_tracks_project_order`)은 적지 않음	08~11번과 같은 이유입니다

6. 시험

`tests/test_track_model.py` (새 파일, 4개): 실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지 (모델의 5칸만)	이름, 순서, `NULL` 허용 여부, `project_id` FK의 `ON DELETE CASCADE`가 같습니다
MASTER 트랙	`order_index`는 0입니다 (DB 기본값). 모델에 없는 칸도 SQL로 읽어 보니 `#7C5CFF`, `0.00`, `0`, `CHAIN`으로 채워져 있었습니다
`track_type`을 빠뜨리면	DB가 거절합니다 (`IntegrityError`)
프로젝트 행을 지우면	트랙 수가 1에서 0이 됩니다 (DB의 `CASCADE`)

`tests/conftest.py`의 `assert_matches_db` 보완: `partial=True`를 주면 모델에 있는 칸과 그 칸에 붙은 FK만 비교합니다. 다른 모델은 그대로 전체를 비교합니다.

전체 시험 70개가 통과했고, `ruff` 검사도 통과했습니다.